DevOps & Linux

Scenario-Based Homework + Quick Q&A

Covers: Linux commands, users & permissions, archiving & cron, networking, AWS EC2, EBS, snapshots, NGINX

How to use this PDF
Part A has 5 scenario-based homework tasks. Each one has a situation, tasks for the student, and a complete **solution with steps and answers** (commands + expected output).
Part B has short, important Q&A for quick revision.

Tip for students: try the tasks yourself first on a Linux machine or an AWS EC2 Ubuntu instance, then compare with the solution.

Scenario list

1. New Dev Team: Users, Groups & Permissions
2. Automated Backup with tar and cron
3. "Website Not Opening": Network Troubleshooting
4. Launch an EC2 Server and Host a Website with NGINX
5. Add Storage: EBS Volume, Mount and Snapshot Backup

Scenario 1: New Dev Team: Users, Groups & Permissions

Situation
TechNova hired two developers, **alice** and **bob**. The admin must give them accounts, put them in a group called **dev_team**, and create a shared folder **/shared-project** that only this team can read and write. A month later bob leaves the company and his access must be removed.

Learning objective
Practice useradd, passwd, groupadd, usermod, chmod, chgrp, passwd -l, userdel (Days 9-12).

Homework Tasks

Task 1. Create the group dev_team.
Task 2. Create users alice and bob with home directories and /bin/bash shell, and set passwords.
Task 3. Add both users to dev_team.
Task 4. Create /shared-project, give group ownership to dev_team, and allow only owner and group full access (others: none).
Task 5. Verify users, group membership and permissions.
Task 6. Bob leaves: lock his account, remove him from the group, then delete him with his home directory.

Solution (Steps and Answers)

Step 1: Create the group.

```
sudo groupadd dev_team
```

Step 2: Create the users and set passwords.

```
sudo useradd -m -s /bin/bash alice
sudo useradd -m -s /bin/bash bob
sudo passwd alice
sudo passwd bob
```

Step 3: Add users to the group (-a appends, so existing groups are kept).

```
sudo usermod -aG dev_team alice
sudo usermod -aG dev_team bob
```

Step 4: Create the shared folder, set group, set permission 770.

```
sudo mkdir /shared-project
sudo chgrp dev_team /shared-project
sudo chmod 770 /shared-project
```

Step 5: Verify.

```
groups alice
ls -ld /shared-project
grep dev_team /etc/group
```

Expected output:

```
alice : alice dev_team
drwxrwx--- 2 root dev_team 4096 Dec 18 10:00 /shared-project
dev_team:x:1003:alice,bob
```

Step 6: Offboard bob.

```
sudo passwd -l bob            # lock account
sudo gpasswd -d bob dev_team   # remove from group
sudo userdel -r bob            # delete user + home
id bob
```

Expected output:

```
id: 'bob': no such user
```

Answer notes
770 = owner rwx, group rwx, others none. Passwords are stored encrypted in /etc/shadow; users in /etc/passwd; groups in /etc/group. A locked account shows **!** in the password field of /etc/shadow.

Scenario 2: Automated Backup with tar and cron

Situation
You are the sysadmin of a small company. The folder **/home/student/project** holds important files. Management wants a compressed backup **every night at midnight**, and backups older than 7 days must be deleted automatically.

Learning objective
Practice tar, gzip, crontab, find and restore (Days 14-15).

Homework Tasks

Task 1. Create the project folder with 3 sample files and a backup folder.
Task 2. Take a manual compressed backup using tar (gzip) and list its contents.
Task 3. Schedule an automatic backup every day at 00:00 using cron.
Task 4. Schedule a cleanup at 01:00 that deletes .tar.gz backups older than 7 days.
Task 5. Restore the backup into /tmp/restore and confirm the files are there.

Solution (Steps and Answers)

Step 1: Prepare folders and files.

```
mkdir -p /home/student/project /home/student/backup
cd /home/student/project
touch file1.txt file2.txt file3.txt
echo "important data" > file1.txt
```

Step 2: Manual backup and verify.

```
tar -czvf /home/student/backup/project_manual.tar.gz -C /home/student project
tar -tzvf /home/student/backup/project_manual.tar.gz
```

Expected output:

```
project/
project/file1.txt
project/file2.txt
project/file3.txt
```

Step 3: Open crontab and add the nightly job. Note: % must be escaped as \% in cron.

```
crontab -e

0 0 * * * tar -czf /home/student/backup/project_$(date +\%F).tar.gz -C /home/student project
```

Step 4: Add the cleanup job in the same crontab.

```
0 1 * * * find /home/student/backup -name "*.tar.gz" -mtime +7 -delete
```

Step 5: Check cron and restore.

```
crontab -l
sudo systemctl status cron
mkdir /tmp/restore
tar -xzvf /home/student/backup/project_manual.tar.gz -C /tmp/restore
ls /tmp/restore/project
```

Expected output:

```
file1.txt  file2.txt  file3.txt
```

Answer notes
Cron fields = minute hour day-of-month month day-of-week. **0 0 * * *** = every day at 00:00. **0 1 * * *** = every day at 01:00. Flags: -c create, -z gzip, -v verbose, -f file, -x extract, -t list, -C change directory.

Scenario 3: "Website Not Opening": Network Troubleshooting

Situation
A user says **https://google.com** does not open on their Linux PC. Their IP is **192.168.1.25**, subnet mask **255.255.255.0**, router (gateway) **192.168.1.1**. Find whether the problem is the local network, the gateway, the internet, or DNS.

Learning objective
Practice IP, subnet mask, gateway, DNS and the commands ip addr, ip route, ping, dig, nslookup, curl (Networking notes).

Homework Tasks

Task 1. Find the IP address and the default gateway of the machine.
Task 2. Write the network part and host part of 192.168.1.25 with mask 255.255.255.0.
Task 3. Test the gateway, then the internet by IP, then DNS, in that order.
Task 4. Explain what each possible result means.
Task 5. Is 192.168.2.10 in the same network? Will the traffic go via the gateway?

Solution (Steps and Answers)

Step 1: Find IP and gateway.

```
ip addr
ip route
```

Expected output:

```
default via 192.168.1.1 dev eth0
(the default gateway is 192.168.1.1)
```

Step 2: Subnet split.
Network part = **192.168.1**, Host part = **25**. All devices 192.168.1.x are in the same network.
Step 3: Test layer by layer.

```
ping -c 3 192.168.1.1     # 1. Is the gateway reachable?
ping -c 3 8.8.8.8         # 2. Is the internet reachable (by IP)?
ping -c 3 google.com      # 3. Does name resolution work?
dig google.com            # DNS check (or: nslookup google.com)
curl https://google.com   # HTTP/HTTPS check
```

Step 4: Read the results.

Result	Meaning / Fix
Gateway ping fails	Local problem: cable/Wi-Fi, wrong IP or wrong gateway. Check ip addr and ip route.
Gateway OK, 8.8.8.8 fails	Router has no internet or ISP issue. Restart router, call ISP.
8.8.8.8 OK, google.com fails	**DNS problem.** Fix DNS server settings (e.g. use 8.8.8.8).
All OK but curl fails	Firewall/proxy or the website itself is blocked or down.

Step 5: Different network?
No. 192.168.2.10 has network part 192.168.2, which is different from 192.168.1. So the packet is sent to the **default gateway** (192.168.1.1) to be routed to the other network.

Flow to remember
Browser asks DNS for the IP, then checks: is the IP in my local network? If No, send to the Gateway, which forwards through the Internet to the server and the response comes back.

Scenario 4: Launch an EC2 Server and Host a Website with NGINX

Situation
A startup needs a small web server on AWS. You must launch an **Ubuntu EC2 instance** (free tier), allow only the needed ports, connect with SSH, install **NGINX** and publish a custom page that anyone on the internet can open.

Learning objective
Practice EC2 launch, security groups, key pairs, SSH, NGINX (AWS Dashboard, EC2, Security Group, NGINX notes).

Homework Tasks

Task 1. Launch an Ubuntu t2.micro instance in your nearest region with a new key pair.
Task 2. Configure the security group: SSH (22) from My IP only, HTTP (80) from anywhere.
Task 3. Connect to the instance using SSH.
Task 4. Install, start and enable NGINX.
Task 5. Replace the default page with your own text and open it in the browser.
Task 6. Stop the instance after practice to avoid charges.

Solution (Steps and Answers)

Step 1: Launch the instance.
EC2 Dashboard > **Launch Instance** > Name: MyFirstInstance > AMI: **Ubuntu Server** > Type: **t2.micro** (free tier) > Key pair: create new, download the **.pem** file > Storage: default 8 GB > Launch.
Step 2: Security group inbound rules.

Type	Protocol	Port	Source
SSH	TCP	22	My IP
HTTP	TCP	80	0.0.0.0/0 (anywhere)

Step 3: Connect with SSH (key file needs strict permission).

```
chmod 400 /path/to/key.pem
ssh -i /path/to/key.pem ubuntu@<Public-IP>
```

Expected output:

```
ubuntu@ip-172-31-x-x:~$
```

Step 4: Install and start NGINX.

```
sudo apt update
sudo apt install nginx -y
sudo systemctl start nginx
sudo systemctl enable nginx
sudo systemctl status nginx
```

Expected output:

```
Active: active (running)
```

Step 5: Create the custom page.

```
echo "<h1>Hello from my EC2 server!</h1>" | sudo tee /var/www/html/index.html
sudo systemctl restart nginx
```

Step 6: Test.

```
curl http://localhost
# Browser: http://<Public-IP>
```

Expected output:

```
<h1>Hello from my EC2 server!</h1>
```

Troubleshooting answers
SSH timeout: port 22 not allowed for your IP in the security group, or instance not running.
Website not loading: port 80 missing in the security group, or NGINX not running.
Permission denied (publickey): wrong username (Ubuntu uses **ubuntu**) or wrong key, or key permission not 400.

Scenario 5: Add Storage: EBS Volume, Mount and Snapshot Backup

Situation
The EC2 server from Scenario 4 is running out of disk space. You need to add a **5 GB EBS volume**, format and mount it at **/mnt/ebs-volume** so it stays mounted after reboot, and then take a **snapshot** as backup.

Learning objective
Practice EBS, fdisk, mkfs, mount, fstab, snapshots and lifecycle policies (EBS and Snapshot notes).

Homework Tasks

Task 1. Create a 5 GB gp3 EBS volume and attach it to the instance.
Task 2. Find the new disk, create a partition and format it with ext4.
Task 3. Mount it on /mnt/ebs-volume and verify.
Task 4. Make the mount permanent using /etc/fstab and test it.
Task 5. Create a manual snapshot, then explain how to automate daily snapshots.

Solution (Steps and Answers)

Step 1: Create and attach the volume.
EC2 > Volumes > **Create volume** (gp3, 5 GB). **Important: choose the same Availability Zone as the instance.** Then Actions > **Attach volume** > select the instance.
Step 2: Find the disk.

```
lsblk
```

Expected output:

```
xvdf   202:80   0   5G  0 disk
```

Step 3: Partition the disk.

```
sudo fdisk /dev/xvdf
# press n  (new partition) -> accept defaults
# press w  (write changes)
```

Step 4: Format the partition.

```
sudo mkfs.ext4 /dev/xvdf1
```

Step 5: Mount and verify.

```
sudo mkdir /mnt/ebs-volume
sudo mount /dev/xvdf1 /mnt/ebs-volume
df -h
```

Expected output:

```
/dev/xvdf1   4.9G   24K   4.6G   1%   /mnt/ebs-volume
```

Step 6: Make it permanent.

```
sudo nano /etc/fstab
# add this line at the end:
/dev/xvdf1   /mnt/ebs-volume   ext4   defaults,nofail   0   2

sudo mount -a      # no error = fstab is correct
```

Step 7: Take a snapshot.
EC2 > Volumes > select volume > Actions > **Create snapshot** > add a description > Create. View it under **Snapshots**.
Step 8: Automate with Data Lifecycle Manager (DLM).
Open **Amazon Data Lifecycle Manager** > Create lifecycle policy > EBS snapshot policy > target volumes by tag (e.g. Environment:Production) > schedule **daily** > retain **7** snapshots > Create.

Answer notes
EBS is block storage attached to **one instance in the same AZ**. For storage shared by many servers use **EFS** (NFS, port 2049, multi-AZ).

Part B: Short & Important Q&A

DevOps and Basics

#	Question	Answer
1	**What is DevOps?**	Culture, practices and tools that bridge Development and Operations for fast, reliable software delivery.
2	**What is CI/CD?**	Continuous Integration (frequent code merge and testing) and Continuous Deployment/Delivery (automatic release).
3	**Name 4 DevOps tools.**	Jenkins, Docker, Kubernetes, Terraform.
4	**Developer vs Tester vs DevOps?**	Developer writes code, Tester validates quality, DevOps manages CI/CD, infrastructure and monitoring.

Linux

#	Question	Answer
5	**Who created Linux?**	Linus Torvalds. It is open-source and derived from Unix ideas.
6	**Parts of Linux architecture?**	Kernel, system libraries, system utilities, shell, applications.
7	**$ vs # in the prompt?**	$ = normal user, # = root user.
8	**Command to see current directory / current user?**	pwd / whoami.
9	**Show hidden files?**	ls -a (hidden files start with a dot).
10	**Where are config files, logs and home folders?**	/etc, /var/log, /home.
11	**Save and quit in Vim? Quit without saving?**	:wq / :q!
12	**Which key enters Insert mode in Vim? Which exits it?**	i enters, Esc exits.
13	**Which file stores encrypted passwords?**	/etc/shadow. Users are in /etc/passwd and groups in /etc/group.
14	**Add a user to a group without removing old groups?**	usermod -aG group username.
15	**Lock a user account?**	passwd -l username (unlock with -u).
16	**What does -rwxr-xr-- mean?**	Owner rwx, group r-x, others r--. Number form: 754.
17	**What does chmod 600 mean?**	Owner read+write, nobody else has access.
18	**What is the link count of a new file?**	1. Each hard link adds 1.
19	**Hard link vs soft link?**	Hard: same inode, cannot cross file systems or link directories. Soft: a pointer, can do both.
20	**sudo vs su?**	sudo runs a command with admin rights (with audit trail). su switches to another user.
21	**Create a gzip tar backup / extract it?**	tar -czvf a.tar.gz dir/ and tar -xzvf a.tar.gz
22	**Cron has how many fields? Run at 2 AM daily?**	5: minute hour day month weekday. 0 2 * * *
23	**Use of grep -i, -r, -v, -n?**	-i ignore case, -r recursive, -v invert match, -n line numbers.
24	**Find files bigger than 100 KB?**	find / -size +100k

Networking

#	Question	Answer
25	**What does a subnet mask do?**	Separates the network part and the host part of an IP address.
26	**What is a default gateway?**	The router that connects your local network to other networks/Internet. See it with ip route.
27	**What does DNS do? Which commands test it?**	Converts domain names to IP addresses. dig, nslookup.
28	**LAN vs MAN vs WAN?**	LAN: building/office. MAN: a city. WAN: countries/world (the Internet).
29	**Name the private IP ranges.**	10.x.x.x, 172.16.x.x to 172.31.x.x, 192.168.x.x.

Cloud and AWS

#	Question	Answer
30	**IaaS, PaaS, SaaS with examples?**	IaaS: EC2. PaaS: Elastic Beanstalk. SaaS: Gmail.
31	**Virtualization vs Cloud?**	Virtualization creates virtual machines on hardware. Cloud delivers on-demand resources over the internet.
32	**Region vs Availability Zone?**	Region = geographic location. AZ = isolated data center inside a region. Use multiple AZs for high availability.
33	**Which instance type is free tier?**	t2.micro (1 vCPU, 1 GB RAM).
34	**SSH and RDP ports?**	SSH 22 (Linux), RDP 3389 (Windows).
35	**Why chmod 400 on the .pem file?**	SSH rejects private keys that others can read.
36	**Default login user for Ubuntu EC2?**	ubuntu.
37	**What is a Security Group?**	A virtual firewall with inbound/outbound rules. Keep SSH restricted to My IP.
38	**On-Demand vs Reserved vs Spot?**	On-Demand: pay as you go. Reserved: 1-3 year commitment, up to 72% off. Spot: up to 90% off but can be interrupted.
39	**Dedicated Host vs Dedicated Instance?**	Host: whole physical server with control and licensing use. Instance: dedicated hardware without that visibility.
40	**What is a Capacity Reservation?**	Reserves capacity in a specific AZ. You pay On-Demand rate even if unused, no discount.
41	**Best EBS type for databases? For backups/archive?**	Provisioned IOPS SSD (io1/io2). Cold HDD (sc1).
42	**EBS vs EFS?**	EBS: block disk for one instance in one AZ. EFS: shared NFS file system, multi-AZ, port 2049.
43	**How to automate EBS snapshots?**	Amazon Data Lifecycle Manager (DLM) policy with schedule and retention.
44	**What is an AMI?**	Template used to launch instances (OS + settings). Can be copied across regions.
45	**Commands to install and start NGINX?**	sudo apt install nginx -y, then sudo systemctl start nginx and enable nginx.